Chapter 05: Function Overloading and Default Arguments

ENG3091

Khiem Nguyen

Contents

1 Function overloading **1**
- 1.1 One job, many types . . . 1
- 1.2 Function overloading . . . 2
- 1.3 Overload resolution . . . 3
- 1.4 Making it compile . . . 3

2 Telling overloads apart **3**
- 2.1 What counts, and what doesn't . . . 3
- 2.2 Type aliases don't count . . . 4
- 2.3 The return type doesn't count . . . 4
- 2.4 Function signatures . . . 4

3 How the compiler picks an overload **5**
- 3.1 Overload resolution, step by step . . . 5
- 3.2 Ambiguous matches . . . 5
- 3.3 Calls that make no sense . . . 6
- 3.4 Deleting a function with `= delete` . . . 6

4 Default arguments **7**
- 4.1 Default arguments . . . 7
- 4.2 Several default arguments . . . 7
- 4.3 The rules (and the reasons) . . . 8
- 4.4 Give the default once… . . . 8
- 4.5 …and before it's used . . . 8
- 4.6 Where the default should go . . . 9
- 4.7 With a header file . . . 10
- 4.8 Default arguments and overloading . . . 10
- 4.9 Watch out: defaults can make calls ambiguous . . . 11

5 Wrapping up **11**
- 5.1 Summary . . . 11
- 5.2 Keep learning . . . 12

1 Function overloading

1.1 One job, many types

Here's a simple function that adds two integers:

```cpp
int add(int x, int y) {
    return x + y;
}
```

What if we also want to add two `doubles`? This `add()` won't do: the arguments would be converted to `int`, and their fractional parts lost.

A workaround is to give every version its own name:

```cpp
int add_integer(int x, int y) { return x + y; }
double add_double(double x, double y) { return x + y; }
```

Now we have to remember all the names. And what about `add(int, double)`, `add(double, int)`, or adding three numbers? The names multiply fast.

1.2 Function overloading

Function overloading lets several functions share one name, as long as their **parameter types** differ. Each one is an **overloaded function**, or *overload* for short.

Listing 1 `function_overloading.cpp`

```cpp
#include <iostream>

int add(int x, int y) { // integer version
    return x + y;
}

double add(double x, double y) { // floating-point version
    return x + y;
}

int main() {
    std::cout << add(1, 2) << '\n';         // add(int, int)
    std::cout << add(42.25, 24.5) << '\n'; // add(double, double)
    return 0;
}
```

```
3
66.75
```

> **Coming from Python**
>
> Python has no overloading: a second `def add` replaces the first. In C++, both exist side by side.

You write `add` every time; the compiler works out which `add` you mean from the arguments.

1.3 Overload resolution

When you call an overloaded function, the compiler matches the call to the right overload by looking at the arguments. This is called **overload resolution**.

Listing 2 `overload_resolution.cpp`

```cpp
#include <iostream>

int add(int x, int y) { return x + y; }
double add(double x, double y) { return x + y; }
double add(double x, double y, double z) { return x + y + z; }

int main() {
    std::cout << add(1, 2) << '\n';     // add(int, int)
    std::cout << add(1.2, 3.4) << '\n'; // add(double, double)

    // add(double, double, double): 1 (an int) and 2.5f (a float)
    // both become doubles
    std::cout << add(1, 2.5f, 1.2) << '\n';
    return 0;
}
```

```
3
4.6
4.7
```

1.4 Making it compile

A program with overloaded functions compiles only if two things are true:

1. **Every overload can be told apart** from the others. *How?* Next section.
2. **Every call can be resolved** to exactly one overload. *How does the compiler choose?* The section after.

Break either rule and you get a compile error. It's just common sense: if *you* couldn't tell which function is meant, neither can the compiler.

2 Telling overloads apart

2.1 What counts, and what doesn't

Property	Tells overloads apart?
Number of parameters	Yes
Types of parameters	Yes (but not type aliases, and not `const` on a value parameter)
Return type	**No**

By **number** of parameters:

```cpp
int add(int x, int y);
int add(int x, int y, int z);
```

By **types** of parameters:

```cpp
int add(int x, int y);
double add(double x, double y);
double add(int x, double y);
double add(double x, int y);
```

2.2 Type aliases don't count

A **type alias** gives an existing type a second name. It's the same type underneath, so it can't tell overloads apart:

```cpp
using Age = int;     // Age is just another name for int
typedef int Height;  // the older way to write the same thing

void print(int value);
void print(Age value);    // same as print(int): not a new overload
void print(Height value); // same as print(int): not a new overload
```

2.3 The return type doesn't count

Listing 3 `return_type_not_enough.cpp`

```cpp
int get_random_value();    // returns an int
double get_random_value(); // error: differs only in return type

int main() { return 0; }
```

```
error: ambiguating new declaration of 'double get_random_value()'
```

Imagine you're the compiler and you see the call `get_random_value()`. Which one would you call? You can't tell from the call, so C++ doesn't allow it.

2.4 Function signatures

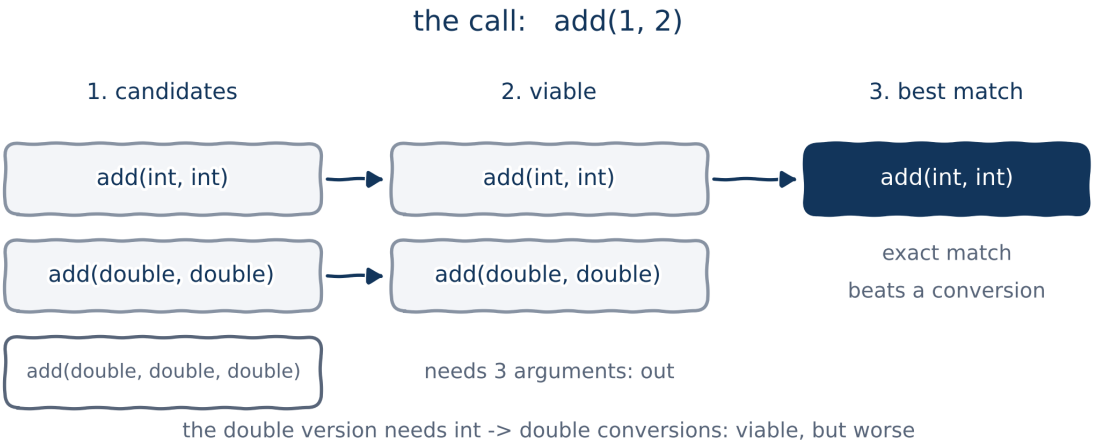
The parts of a function that tell overloads apart are called its **signature**: the function's **name** plus the **number and types of its parameters**. The return type is *not* part of it.

Declaration	Signature
`int add(int x, int y);`	`add(int, int)`
`double add(double x, double y);`	`add(double, double)`
`double add(int a, double b, int c);`	`add(int, double, int)`

Every overload must have a different signature. Parameter names don't matter either: `add(int x, int y)` and `add(int a, int b)` have the same signature.

3 How the compiler picks an overload

3.1 Overload resolution, step by step



1. **Candidates:** every function with that name.
2. **Viable functions:** those with the right number of parameters, where each argument can be converted to the parameter type.
3. **Best match:** an exact match beats a promotion, which beats a conversion. If one function is strictly better than the rest, it's called.

3.2 Ambiguous matches

If two overloads are *equally* good, the compiler refuses to guess:

Listing 4 `ambiguous_match.cpp`

```cpp
#include <iostream>

void print(int x) { std::cout << "int " << x << '\n'; }
void print(double x) { std::cout << "double " << x << '\n'; }

int main() {
    print(5);   // exact match: print(int)
    print(5.0); // exact match: print(double)
    print(5L);  // a long: converts to int AND to double. Which?
    return 0;
}
```

```
error: call of overloaded 'print(long int)' is ambiguous
```

`5L` is a `long`. Converting it to `int` and converting it to `double` are equally good, so neither wins. The fix: pass an argument of the type you mean, like `5` or `5.0`.

3.3 Calls that make no sense

Listing 5 `print_int_surprise.cpp`

```cpp
#include <iostream>

void print_int(int x) { std::cout << x << '\n'; }

int main() {
    print_int(5);    // prints 5
    print_int('a');  // prints 97: does this make sense?
    print_int(true); // prints 1: does this make sense?
    return 0;
}
```

```
5
97
1
```

A `char` and a `bool` both convert to `int` silently: `'a'` is stored as the number 97. Suppose that printing a character or a Boolean as a number makes no sense for us. Can we forbid it?

3.4 Deleting a function with `= delete`

Listing 6 `delete_function.cpp`

```cpp
#include <iostream>

void print_int(int x) { std::cout << x << '\n'; }

void print_int(char) = delete; // calling these is now an error
void print_int(bool) = delete;

int main() {
    print_int(97);  // fine
    print_int('a'); // error: use of deleted function
    return 0;
}
```

```
error: use of deleted function 'void print_int(char)'
```

- `= delete` declares an overload that **must not be called**. Overload resolution still picks it for a `char` argument, and then the compiler reports an error.
- Careful: `print_int(5.0)` is now an error too, because a `double` converts equally well to `int`, `char` and `bool`, so the call is ambiguous.

4 Default arguments

4.1 Default arguments

A **default argument** is a value a parameter gets when the caller doesn't supply one.

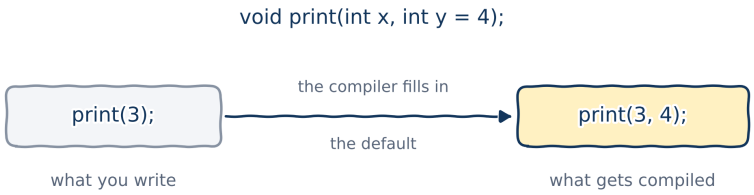
Listing 7 `default_argument.cpp`

```cpp
#include <iostream>

void print(int x, int y = 4) { // 4 is the default argument
    std::cout << "x: " << x << ", y: " << y << '\n';
}

int main() {
    print(1, 2); // y uses the argument we give: 2
    print(3);    // y uses the default: same as print(3, 4)
    return 0;
}
```

```
x: 1, y: 2
x: 3, y: 4
```



4.2 Several default arguments

Listing 8 `multiple_default_arguments.cpp`

```cpp
#include <iostream>

void print_numbers(int x = 10, int y = 20, int z = 30) {
    std::cout << x << ' ' << y << ' ' << z << '\n';
}

int main() {
    print_numbers(1, 2, 3); // no defaults used
    print_numbers(1, 2);    // z defaulted
    print_numbers(1);       // y and z defaulted
    print_numbers();        // all three defaulted
    return 0;
}
```

```
1 2 3
1 2 30
1 20 30
10 20 30
```

Arguments are matched to parameters from the **left**, and the defaults fill in the rest from the right.

4.3 The rules (and the reasons)

C++ has no syntax like `print_numbers( , , 3)`. Everything else follows from that:

- **You can't skip an argument.** To give `z` a value, you must also give `x` and `y`.
- **Defaults must come last.** Once a parameter has a default, every parameter to its right needs one too:

```
1 void print(int x = 10, int y); // error: y needs a default as well
```

- **Put the parameter callers most often change first** among the defaulted ones.

> **i** Coming from Python
>
> Python lets you name arguments in the call: `print_numbers(z=3)`. C++ has no named arguments, so the order of parameters matters much more.

4.4 Give the default once…

A function with a forward declaration can have its default in the declaration *or* in the definition, but **not both**:

Listing 9 `default_argument_twice.cpp`

```
1  #include <iostream>
2
3  void print(int x, int y = 4); // forward declaration with default
4
5  void print(int x, int y = 4) { // error: default given again
6      std::cout << "x: " << x << ", y: " << y << '\n';
7  }
8
9  int main() {
10     print(3);
11     return 0;
12 }
```

```
error: default argument given for parameter 2 of 'void print(int, int)'
```

4.5 …and before it's used

The compiler reads top to bottom (Chapter 02!), so it must see the default before the call:

Listing 10 `default_argument_too_late.cpp`

```cpp
#include <iostream>

void print(int x, int y); // forward declaration: no default here

int main() {
    print(3); // error: the compiler doesn't know about the default yet
    return 0;
}

void print(int x, int y = 4) {
    std::cout << "x: " << x << ", y: " << y << '\n';
}
```

```
error: too few arguments to function 'void print(int, int)'
```

4.6 Where the default should go

Put the default in the **forward declaration**:

Listing 11 `forward_declaration_default_arguments.cpp`

```cpp
#include <iostream>

void print(int x, int y = 4); // the default goes here...

int main() {
    print(3);
    return 0;
}

void print(int x, int y) { // ...and not here
    std::cout << "x: " << x << ", y: " << y << '\n';
}
```

```
x: 3, y: 4
```

> Best practice
>
> Default in the forward declaration if there is one (especially in a header); otherwise in the definition.

4.7 With a header file

Listing 12 `print.h`

```
#ifndef PRINT_H
#define PRINT_H

void print(int x, int y = 4); // default

#endif
```

Listing 13 `main.cpp`

```
#include "print.h" // + its default

int main() {
    print(5); // print(5, 4)
    return 0;
}
```

Listing 14 `print.cpp`

```
#include "print.h"
#include <iostream>

void print(int x, int y) { // no default repeated here
    std::cout << "x: " << x << ", y: " << y << '\n';
}
```

```
$ g++ -std=c++20 -Wall main.cpp print.cpp -o program
$ ./program
x: 5, y: 4
```

4.8 Default arguments and overloading

Functions with default arguments can be overloaded:

Listing 15 `default_argument_overloaded.cpp`

```cpp
#include <iostream>
#include <string>

void print(std::string s) { std::cout << s << '\n'; }
void print(char c = '*') { std::cout << c << '\n'; }

int main() {
    print("Hello, world"); // print(std::string)
    print('a');            // print(char)
    print();               // print(char), using the default '*'
    return 0;
}
```

```
Hello, world
a
*
```

4.9 Watch out: defaults can make calls ambiguous

A default value is **not** part of the signature, so these are two different overloads. But a call can still match both:

Listing 16 `default_argument_ambiguous.cpp`

```cpp
#include <iostream>

void print(int x) { std::cout << x << '\n'; }
void print(int x, int y = 10) { std::cout << x << ' ' << y << '\n'; }

int main() {
    print(1, 2); // fine: only print(int, int) takes two arguments
    print(1);    // error: print(int) or print(int, int = 10)?
    return 0;
}
```

```
error: call of overloaded 'print(int)' is ambiguous
```

5 Wrapping up

5.1 Summary

- **Overloading:** several functions can share a name if their **signatures** (name, number and types of parameters) differ. The return type doesn't count.
- **Overload resolution:** the compiler picks the best match; an exact match beats a conversion. If two overloads tie, the call is **ambiguous** and won't compile.
- `= delete` forbids particular calls.
- **Default arguments** fill in parameters the caller leaves out. Defaults must be the rightmost parameters, and can't be skipped.
- Give a default **once**, in the forward declaration (header) if there is one, and before it's used.

- Defaults are not part of the signature, so they can make calls ambiguous.

5.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Function overload resolution and ambiguous matches. The exact ranking rules are long; try asking Gemini: *"Please teach me function overload resolution and ambiguous matches in C++."*
2. Implicit type conversion, and the difference between promotions and conversions
3. Type aliases

Next up, Chapter 06: function templates. Overloading means writing every version by hand. What if the compiler could write them for us?